

Corey Montella, Aung Kant Ko, Steven McPhillimey

- Rust's memory safety, performance and low-level control make it a strong foundation for robotics. Numerical kernels can still become lengthy and difficult to verify when each backend requires specialized code.
- Mech is a numerical language implemented in Rust and embeddable in Rust applications. Its compiler specializes typed equations without backend-specific source annotations. A bearing-only EKF illustrates its numerical and reactive features, which also apply beyond state estimation.

Rust applications can embed Mech source through its Rust library, or load a compiled dylib. Both interfaces bind named inputs and retain state between updates. C++ and Python bindings are planned.

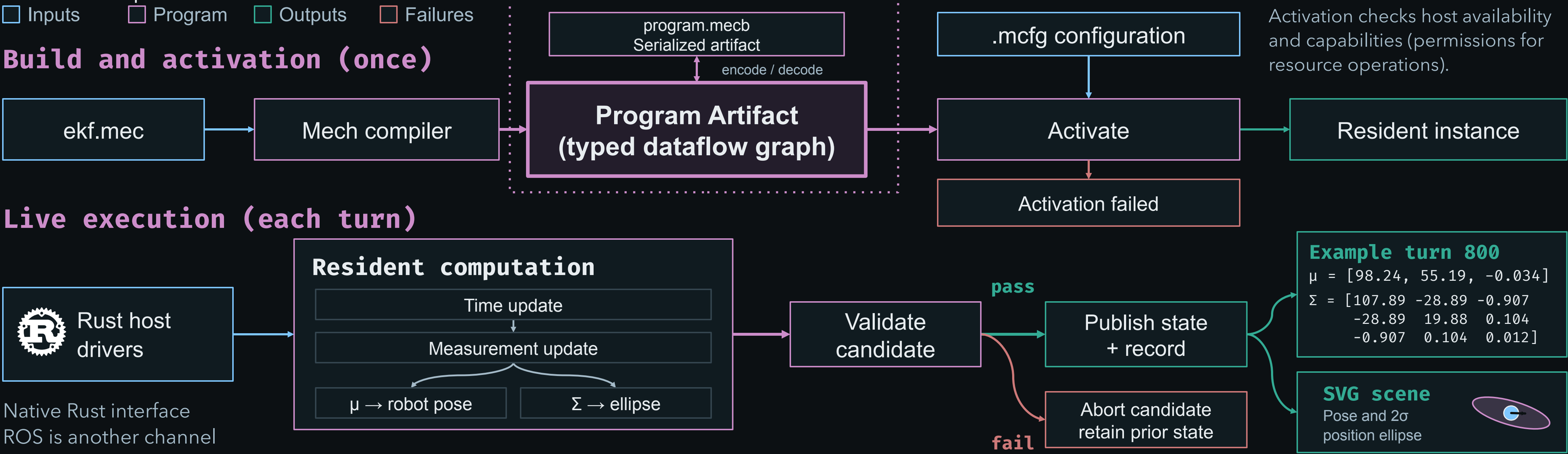
```
use mech::kernel::{Backend, Kernel};
let source = include_str!("ekf.mec");
let kernel = Kernel::from_source(source)
    .input("bearing", [-0.55; 4])
    .export("state")
    .compile(Backend::Jit)?;
let mut ekf = kernel.start()?;
ekf.turn([("bearing", [-0.54; 4])]);
let state = ekf.state("state");
```

Compile once, then submit successive measurement updates. Each checked turn retains the last accepted state.

Artifact	Bytes	M/s $\pm$ MAD	Peak RSS (MiB)
Mech scalar	33,544	14.671 $\pm$ 0.008	2.69 $\pm$ 0.00
Rust scalar	50,016	21.121 $\pm$ 0.005	2.69 $\pm$ 0.00
Mech SIMD-4	33,864	34.863 $\pm$ 0.010	2.69 $\pm$ 0.00

```
let kernel = unsafe { Kernel::load_bundle("ekf.bundle")? };
let mut ekf = kernel.start()?;
ekf.turn([("bearing", [-0.54; 4])]);
```

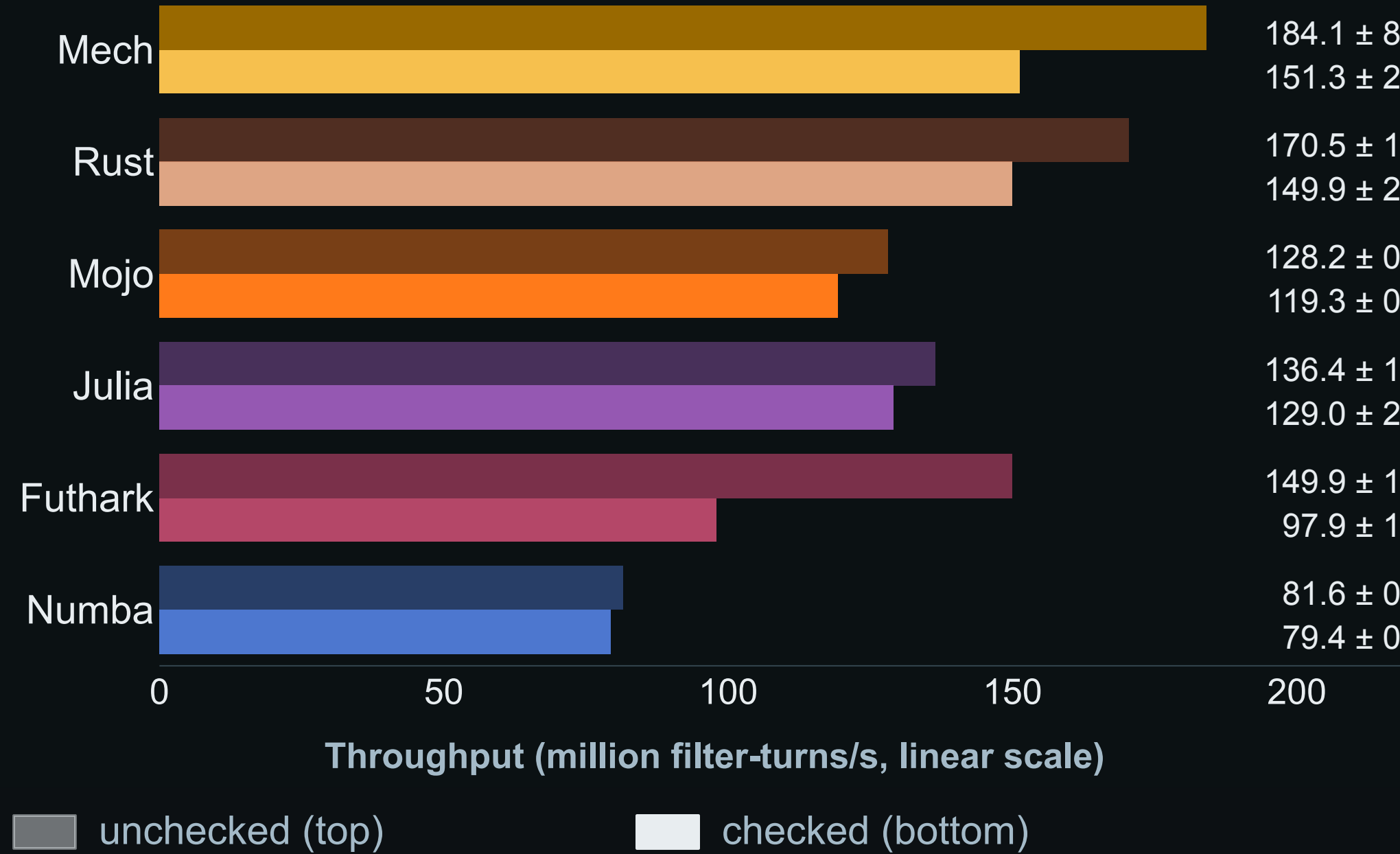
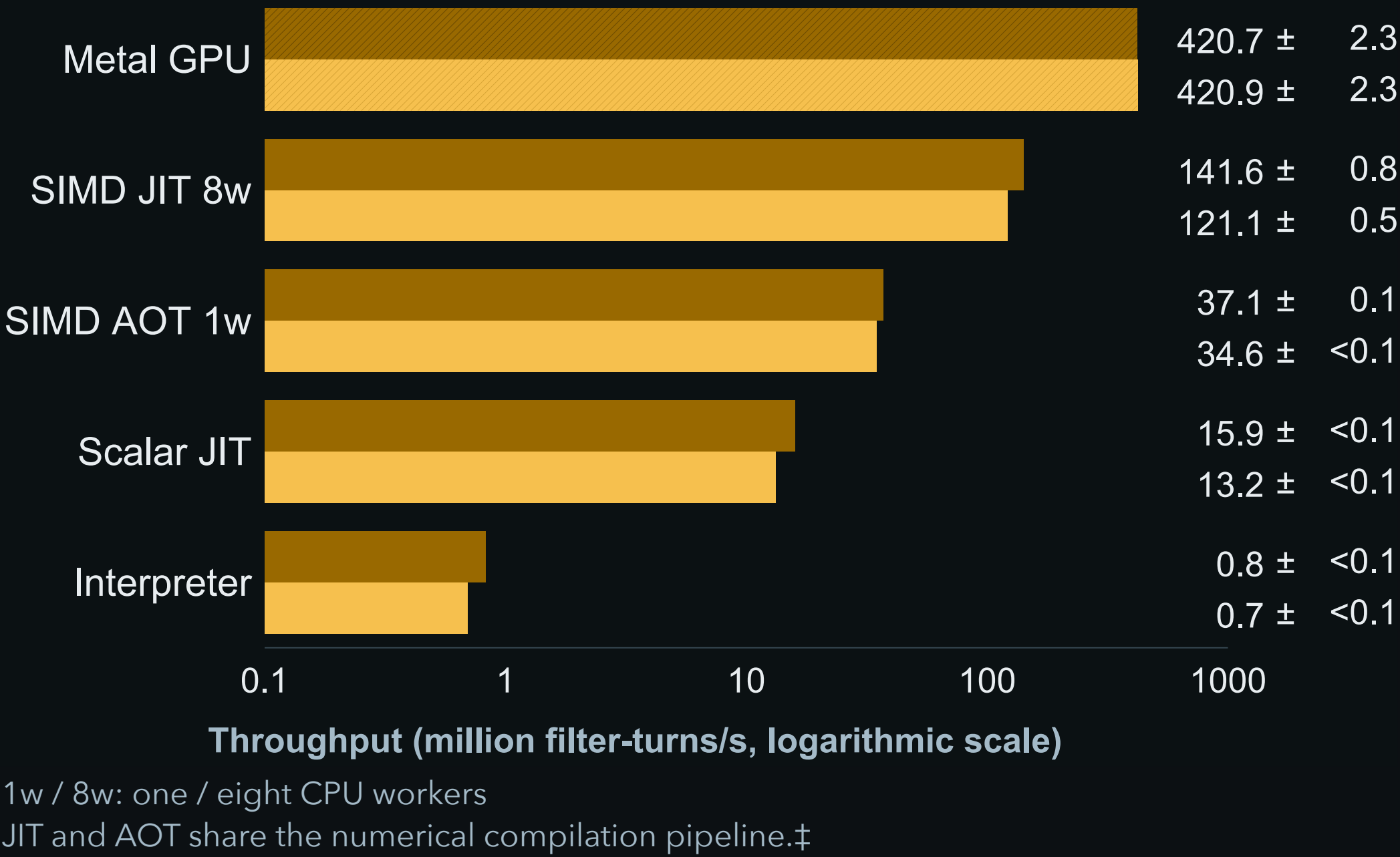
Robots commonly use feedback control loops over continuous data streams. Mech handles successive updates through its native Rust interface or ROS channels. Each turn validates candidate state against integrity constraints. Rejection notifies the caller and retains the previous state.



Mech's typed, declarative IR supports interpretation, scalar/SIMD CPU, multicore, GPU and WebAssembly/WGPU from the same source.

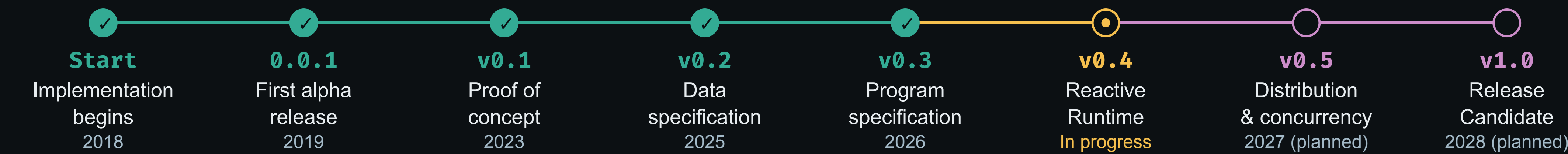
The Mech rows use the same f32 EKF source shown at right.

Unchecked runs omit integrity predicates and retain per-turn publication in the Mech chart. Checking reduces SIMD AOT throughput by 6.6%. The Metal difference is within measured variation.

8 workers, fused execution, median $\pm$ MADPer-turn publication, median $\pm$ MAD

Mech is in beta. Further validation is needed beyond the EKF case study.

Over the next year, we plan evaluation across real robot hardware platforms and in robot deployments.



Experience this poster as a live reactive program. Mech's WebAssembly runtime supports the CPU and WGPU comparison online.



Benchmark setup. Apple M1: 4P + 4E CPU, 8-core GPU, 8GB, macOS 15.6.1. Charts: 332, 500, 5000 filters × 40 turns, n=10. Median ± MAD.
Run: 1.96 s nightly ecd8f3da3 / LLVM 22.1.0, Cranelift 0.131.3, Julia 1.12.6, Mojo 1.1.0 / MAX 26.6.0, Futhark 0.27.1 / ISPC 13.1.0 / LLVM 22.1.8. Python 3.12.14 / NumPy 2.2.6 / Numba 0.61.2, Apple clang 17.0.0.
Mech v0.4 development (package 0.3.6). Backend pairs f4b69052c, GPU eedc1c75b, dylib8d98af1d, ios/workshop-benchmarks, benchmarks/iros-2026.
† DLib table: minimal ABI layout, 1 thread, 10,000 filters × 200 turns, n=7. RSS covers the process. Compile/loop/warmup excluded. File sizes reflect linker/segment layout. The new embedding wrapper is not timed.
‡ SIMD JIT and AOT share the Cranelift pipeline. JIT compiles on demand. AOT emits a library ahead of deployment, allowing the host to omit the compiler.
[1] Thrun, Burgard & Fox. Probabilistic Robotics. MIT Press, 2005. §7.4.2, Table 7.2 (adapted).

Mech offers a concrete syntax that expresses numerical algorithms as typed matrix equations. The bearing-only EKF below adapts the textbook formulation [1], preserving its matrix-level notation.

SIMD Rust uses almost five times as many normalized source characters here, mainly for SIMD and batch support. Reusing the Mech source across backends reduces the specialized implementations to maintain.

Compilation produces a typed dataflow artifact. Matrix operations use nalgebra in the ordinary backend; specialized backends compile the graph directly.

```
+> math/*
Δt := 0.1
m := [140 12]'
Q := [0.01 0
      0 0.0025]
R := 0.25
I := [1 0 0
      0 1 0
      0 0 1]
~μ := [55 25 0.4]'
~Σ := [100 0 0
      0 100 0
      0 0 0.15]

-- ~ mutable
-- := define
-- = assign
-- ** matrix multiply
-- ' transpose
-- * broadcast multiply
```

```

θ := μ[3]
d := v * Δt
G := [1 0 (0 - d * sin(θ))
      0 1 d * cos(θ)
      0 0 1]
V := [cos(θ) * Δt 0
      sin(θ) * Δt 0
      0 Δt]

```

$$\begin{aligned} \bar{\mu} &:= \mu + \begin{bmatrix} d * \cos(\theta) \\ d * \sin(\theta) \\ \omega * \Delta t \end{bmatrix} \\ \bar{\Sigma} &:= G ** \Sigma ** G' + V ** Q ** V \end{aligned}$$

```

δ := m - μ[1..=2]
δx := δ[1]
δy := δ[2]
q := δx ^ 2 + δy ^ 2

```

```

 $\hat{z} := \text{atan2}(\delta y, \delta x) - \bar{\mu}[3]$ 
 $r := z - \hat{z}$ 
 $v := \text{atan2}(\sin(r), \cos(r))$ 

```

```
H := [δy / q, (0 - δx) / q, -1]
S := H ** ξ ** H' + R
K := (ξ ** H') / S[1]
A := I - K ** H
```

$$\begin{aligned} \mu_+ &:= \bar{\mu} + K \star v \\ \Sigma_+ &:= A \star \bar{\Sigma} \star A' + (K \star K') \star R \\ \mu &= \mu_+ \\ \Sigma &= \Sigma_+ \end{aligned}$$

Application source	Mech	Rust textbook	Rust SIMD
Setup	340	486	561
EKF + candidate update	356	1,024	1,119
Validation	383	263	396
Matrix helpers	0	251	310
SIMD + batch support	0	331	2,857
Total	1,079	2,355	5,243

The time and measurement updates remain explicit matrix equations. Backend code handles vector layout and batching, so the same numerical source can serve scalar and SIMD execution.

Rust's selected SIMD implementation includes vector adapters and a batch dispatcher with rollback. Mech delegates these operations to reusable compiler and runtime components, which explains its zero entries for helpers and SIMD support.

* Programmer-chosen names count as one character per occurrence. Comments and nonliteral whitespace are removed. General library, compiler and runtime internals are excluded.

